

32548 Cyber Security - Week 5 Lab: Understanding TCP/IP based Attacks

32548 Cyber Security – Week 5 Lab: Understanding TCP/IP based Attacks

Name: Jinkyung Kim

Student ID: 14657314

Tutorial: Tut03 | **Subject:** 32548 | **Group:** Cmp1

Date: 2 September 2026

Task 1 – SYN Flooding Attack

SYN flooding is where an attacker floods the victim machine with TCP SYN packets, but doesn't complete the 3-way handshake. As a result, the victim machine will have lots of half-open connections sitting in its queue (connections which sent a SYN, received a SYN-ACK, but never received the final ACK) and once its queue is full the machine can no longer accept connections. I ran netwag tool 76 (Synflood) from the attacker to target the client (10.0.2.8) on port 80 and captured it on the client with tshark (need tshark here instead of wireshark since the flood is so intensive wireshark will crash).

76 --dst-ip 10.0.2.8 --dst-port 80 --spooftip "best"

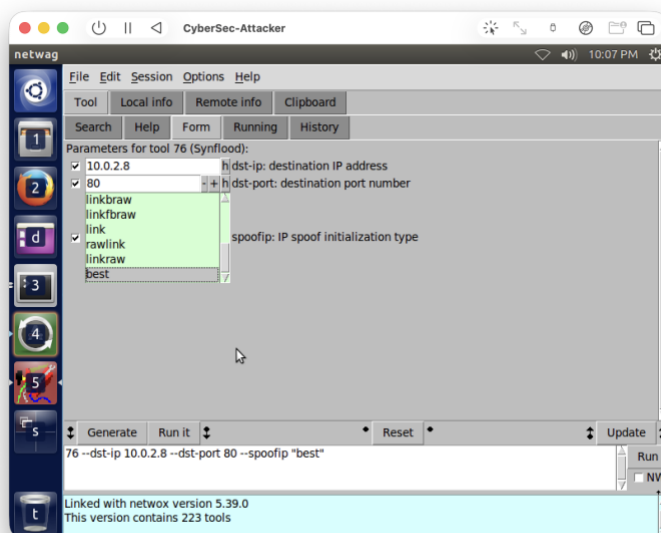


Fig 1. netwag tool 76 (Synflood) targeting the client 10.0.2.8 on port 80, with IP spoofing set to "best".

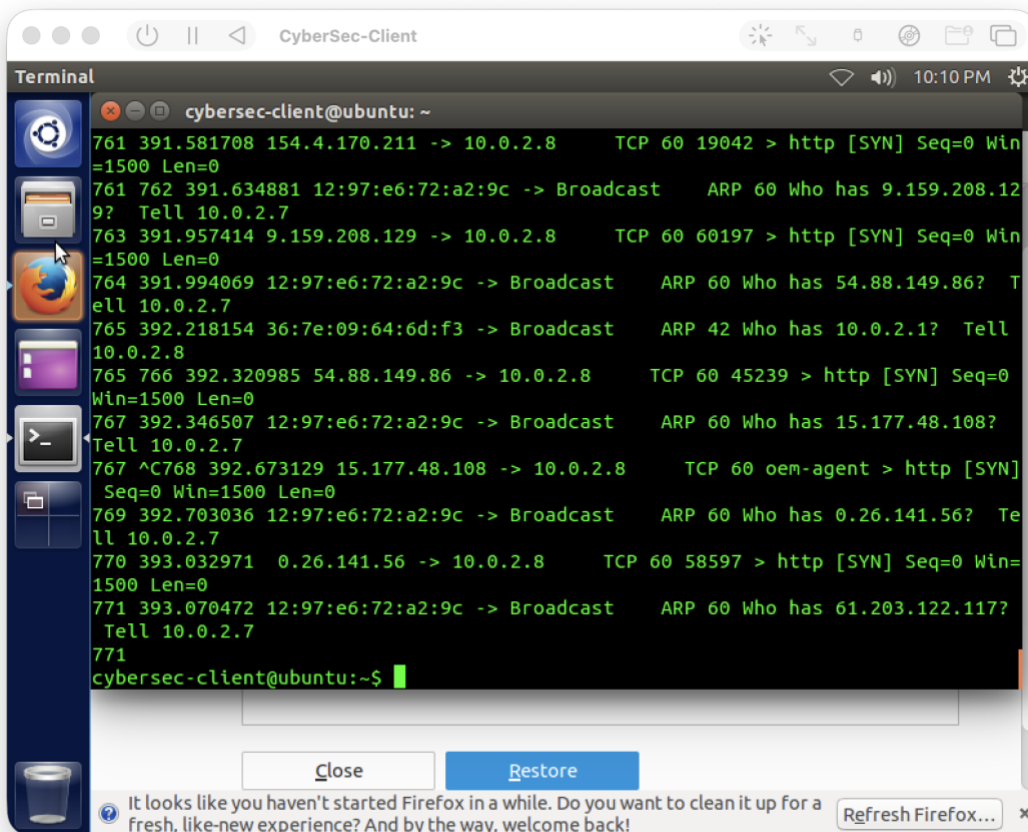


Fig 2. The client's tshark capturing the attack - a flood of TCP [SYN] packets going to 10.0.2.8:http, each from a different random spoofed source IP.

Observed Results: after i executed the attack, my client's tshark capture showed massive amounts of TCP [SYN] packets directed to 10.0.2.8:80, each with a randomized source IP from due to the IP spoofing. Every packet fails to complete the TCP handshake. You will also notice some ARP "who has ..?" requests from 10.0.2.7 woven in there as well, which is the actual attacking machine trying to resolve those randomized IP addresses.

Severity/consequence: this attack is classified as a Denial of Service (DoS) attack and I would consider the severity to be high. Each spoofed SYN packet causes the victim to establish a half-open connection table entry and send a SYN-ACK response. However, because the spoofed source will not send an ACK to respond to the SYN-ACK, the connection just waits until it times out. Once the backlog queue gets full, new legitimate connections cannot be accepted by the victim, attacking the Availability portion of the service. That about sums up why a SYN flood is considered a DoS.

Task 2 – ARP Cache Poisoning

ARP cache poisoning takes advantage of ARP being stateless - a host will accept any ARP reply that it receives and update its cache with that information even if it did not broadcast an ARP request. Therefore an attacker can send out forged ARP replies and cause the victim machine to associate an IP address with the wrong MAC address. I used the netmag tool 80 (Periodically send ARP replies) to poison the server by tricking it into thinking that the client (10.0.2.8) had the MAC address aa:bb:cc:dd:ee:ff.

```
80 --eth aa:bb:cc:dd:ee:ff --ip 10.0.2.8 --device "Eth0" --eth-dst 66:60:18:56:2b:9c --ip-dst 10.0.2.6 --sleep 1000
```

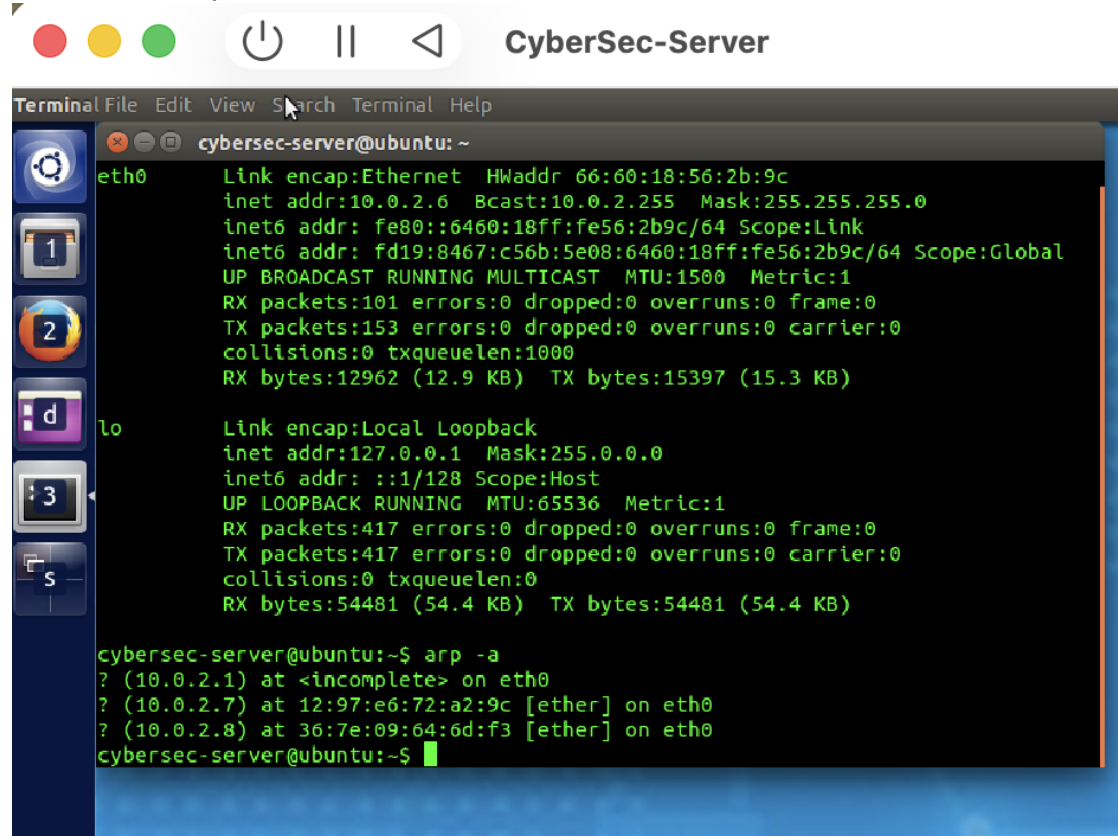


Fig 3. The server's ARP table BEFORE the attack - 10.0.2.8 is correctly mapped to the real client MAC 36:7e:09:64:6d:f3.

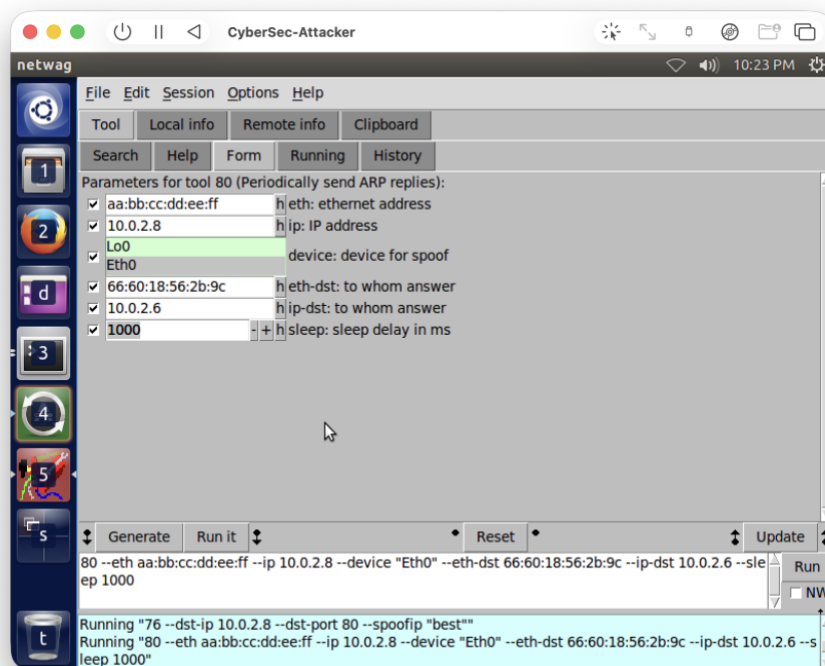


Fig 4. netwag tool 80 running, sending the forged ARP replies (10.0.2.8 is at aa:bb:cc:dd:ee:ff) to the server.

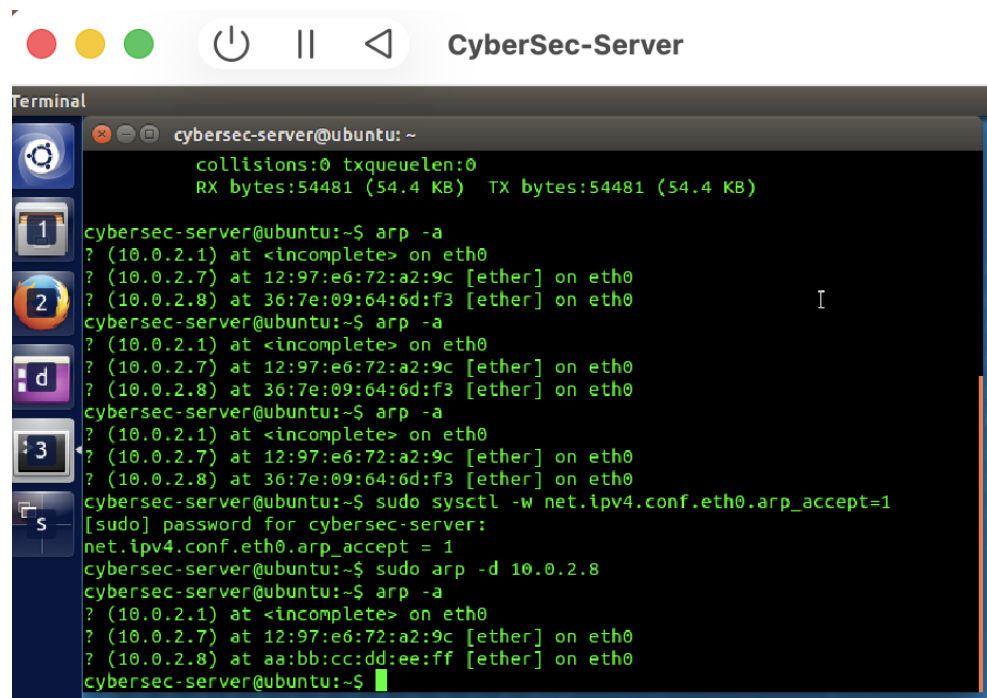


Fig 5. The server's ARP table AFTER the attack - 10.0.2.8 is now poisoned to the fake MAC aa:bb:cc:dd:ee:ff.

Observed Results: before launching the attack my server had 10.0.2.8 correctly mapped to the real client's MAC (36:7e:09:64:6d:f3). After I executed tool 80, the ARP table on the server got updated so now 10.0.2.8 points to the attacker controlled aa:bb:cc:dd:ee:ff MAC address. Something interesting to note is that at first it wouldn't update - Modern Linux actually protects an existing (reachable) arp cache entry by ignoring unsolicited replies. Exactly the type of anti-poisoning defence I was talking about. I had to enable net.ipv4.conf.eth0. arp_accept=1 and delete the stale entry on the server first, before the forged reply was accepted and cache poisoned.

Severity/consequence: Use static ARP entries so that the mapping cannot be overwritten by reply; use Dynamic ARP Inspection (DAI) on the switch to validate ARP messages against a DHCP snooping table; leave the OS-level protection enabled (the arp_accept behaviour I had to disable is actually a defence mechanism on by default); and monitor the network with arpwatsh. Encrypting traffic also helps, so even if it gets redirected it's useless to the attacker.

Task 3 – ICMP Redirect Attack

ICMP redirect message should usually be used by router to inform the host "instead of me for this destination use that gateway". The issue is that host does not authenticate it properly where it comes from. Attacker can send ICMP redirect forged packet and successfully poison the victim machine routing table. I used tool netwag 86 and capture it on client with wireshark(filter icmp).

86 --device "Eth0" --gw 10.0.2.7 --spoofip "best" --src-ip 10.0.2.6

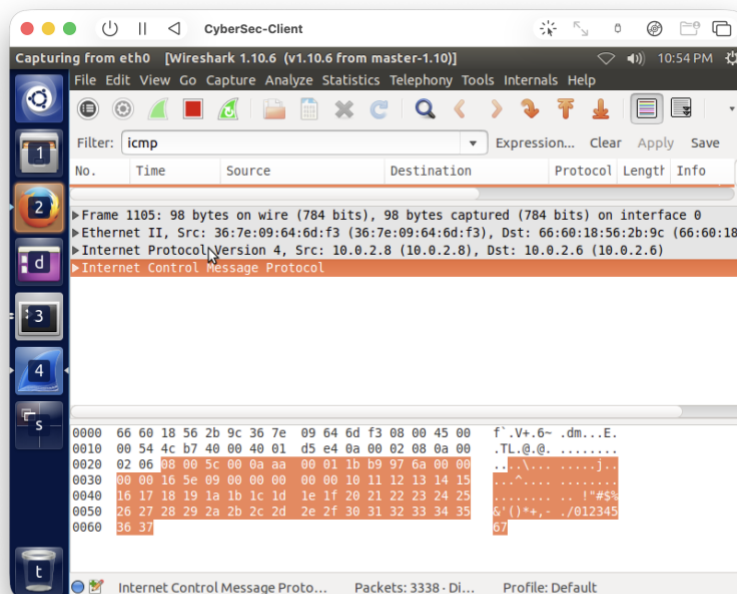


Fig 6. The client pinging the server (10.0.2.8 → 10.0.2.6), captured in wireshark as ICMP Echo (ping) requests - the normal traffic before the attack.

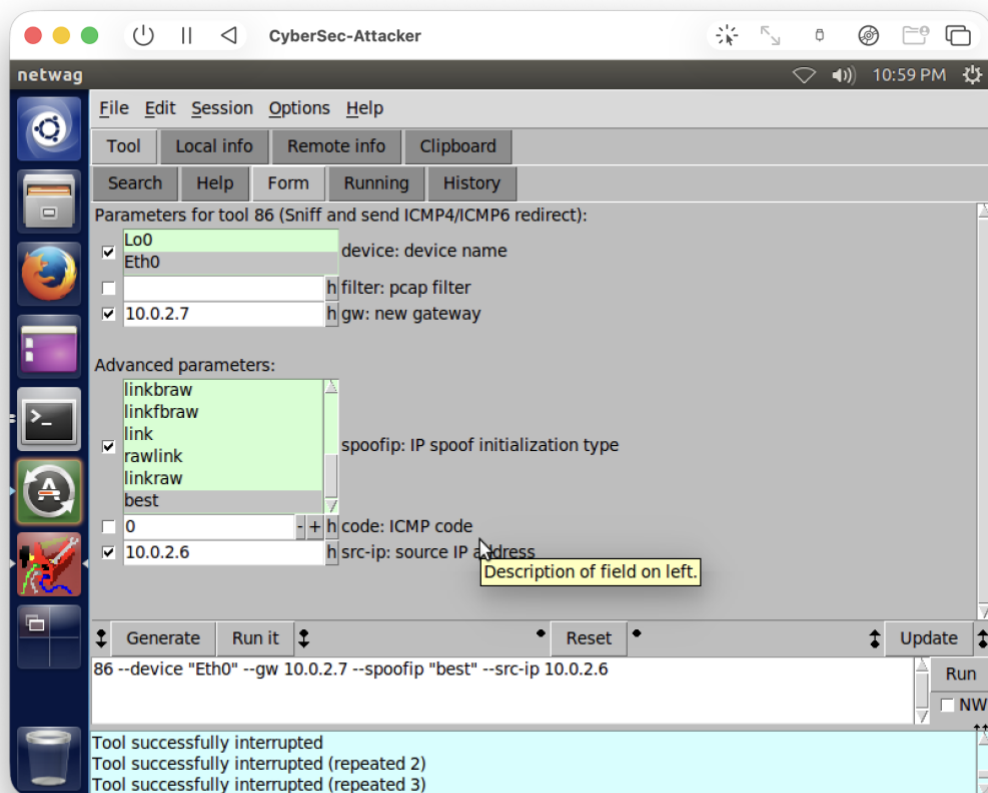


Fig 7. netwag tool 86 configured to send a forged ICMP redirect, with the new gateway set to 10.0.2.7 (the attacker) and the source set to 10.0.2.6.

Making the attack actually work on a switched network

Originally no redirect appeared at all. Reason is the same as with the vmnet thing i described - tool 86 has to SNIFF the client's ping to the server, then react by sending a redirect. On this switched virtual network however the attacker couldn't see client->server unicasts, so it had nothing to react to. Workaround for this was that i ARP-poisoned the client so it think that server (10.0.2.6) was at the attackers real MAC, causing it to send its ping through the attacker. Then attacker could see the ping and tool 86 successfully.

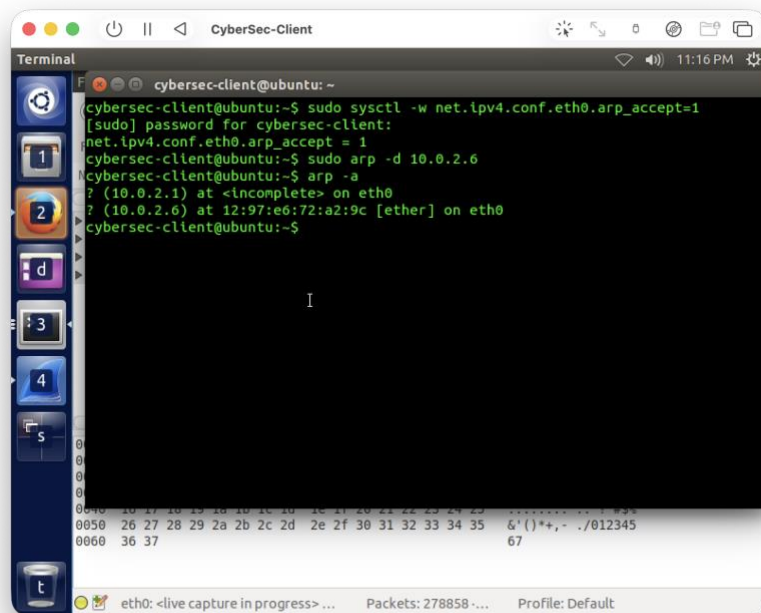


Fig 8. ARP poisoning the client so 10.0.2.6 (the server) now point to the attacker's MAC 12:97:e6:72:a2:9c - this let the attacker sniff the client's ping.

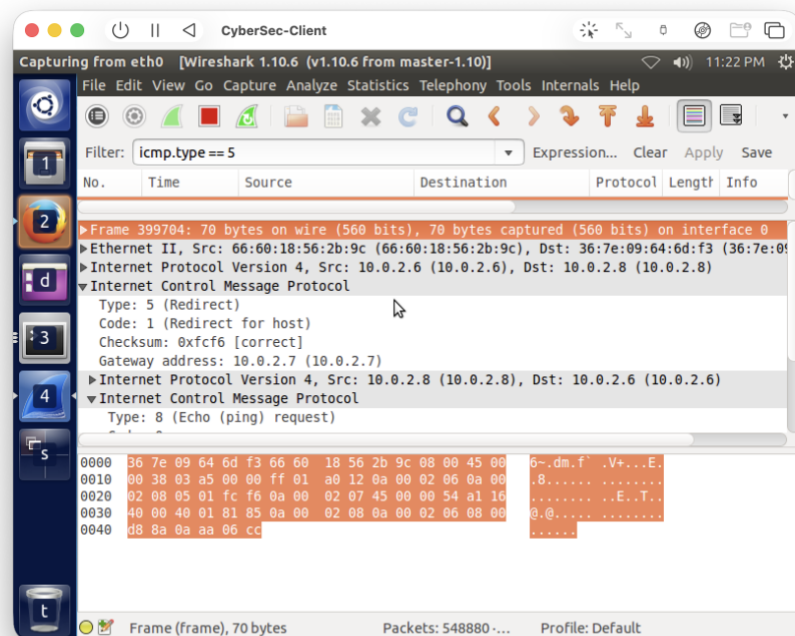


Fig 9. The forged ICMP Redirect captured on the client (filter `icmp.type == 5`): Type 5 (Redirect), Code 1 (Redirect for host), Gateway address 10.0.2.7 (the attacker).

Observed Results: After ARP trick, the client received a spoofed ICMP Redirect (Type 5 Code 1 "redirect for host") message with a source address of 10.0.2.6, instructing it to use gateway 10.0.2.7 (attacker host) to access 10.0.2.6. Should the client accept this redirect, all traffic to 10.0.2.6 will be routed through the attacker thus creating a man-in-the-middle attack.

Severity/consequence: Disable accepting ICMP redirects on the hosts (`net.ipv4.conf.all.accept_redirects=0` and `net.ipv4.conf.all.secure_redirects=0`), filter ICMP redirect messages on the firewall, and use static routes so the routing cannot be changed by a message. Also, since the attack required ARP poisoning to even work in the first place, all of the ARP defenses from Task 2 (static ARP entries, Dynamic ARP Inspection) help mitigate this as well.